

Explain name="output" usage

1. explain why we used name="output" in 3 places: <form method="GET"> <label for="input1">Input 1 :
 </label> <input type="number" name="input1" value="<?php echo \$input1 ?>">

 <label for="input2">Input 2 : </label> <input type="number" name="input2" value="<?php
 echo \$input2 ?>">

 <label for="output">Output : </label> <input type="number" name="output"
 value="<?php echo \$output ?>">

 <input type="submit" name="output" value="add"> <input
 type="submit" name="output" value="sub"> </form>.....2



1. explain why we used name="output" in 3 places:

```
<form method="GET">
<label for="input1">Input 1 : &nbsp;&nbsp;&nbsp;</label>
<input type="number" name="input1" value="<?php echo $input1 ?>">
<br>
<br>
<label for="input2">Input 2 : &nbsp;&nbsp;&nbsp;</label>
<input type="number" name="input2" value="<?php echo $input2 ?>">
<br>
<br>
<label for="output">Output : </label>
<input type="number" name="output" value="<?php echo $output ?>">
<br>
<br>

<input type="submit" name="output" value="add">
<input type="submit" name="output" value="sub">
</form>
```

Good question 👍

Let's break it down. In your form, you used name="output" in **three different places**:

```
html

<input type="number" name="output" value="<?php echo $output ?>">
<input type="submit" name="output" value="add">
<input type="submit" name="output" value="sub">
```

Here's why that was done:

1. <input type="number" name="output">

- This input box is where the *calculated result* (addition or subtraction of **input1** and **input2**) is displayed.
- Giving it name="output" means when the form is submitted, its value is also sent to the server as `$_GET['output']`.

2. `<input type="submit" name="output" value="add">`

3. `<input type="submit" name="output" value="sub">`

- These two are **buttons**, not result fields.
- Both share the same `name="output"`, but their `value` is different (`add` or `sub`).
- When you press one of them, only that button's value is sent to the server.
 - If you clicked **add**, then `$_GET['output'] = "add"`.
 - If you clicked **sub**, then `$_GET['output'] = "sub"`.

This way, your PHP code can check:

php

```
if ($_GET['output'] == "add") {
    $output = $input1 + $input2;
} elseif ($_GET['output'] == "sub") {
    $output = $input1 - $input2;
}
```

🔑 Why the same `name="output"` was reused?

Because the **submit buttons and the output box are logically connected**:

- The buttons tell PHP *what operation to perform* (add or sub).
- The number field with the same name displays the *result* of that operation.

It's a shortcut — instead of using different names (like **operation** for buttons and **output** for result), everything was put under **output**.

But this can be a bit confusing and isn't the cleanest practice.

✅ A **clearer way** would be:

html

```
<input type="number" name="result" value="<?php echo $output ?>">
<input type="submit" name="operation" value="add">
<input type="submit" name="operation" value="sub">
```

Then in PHP:

php

```
if ($_GET['operation'] == "add") {  
    $output = $_GET['input1'] + $_GET['input2'];  
} elseif ($_GET['operation'] == "sub") {  
    $output = $_GET['input1'] - $_GET['input2'];  
}
```

👉 Do you want me to **rewrite your code in this cleaner version** (separating **operation** and **output**)?